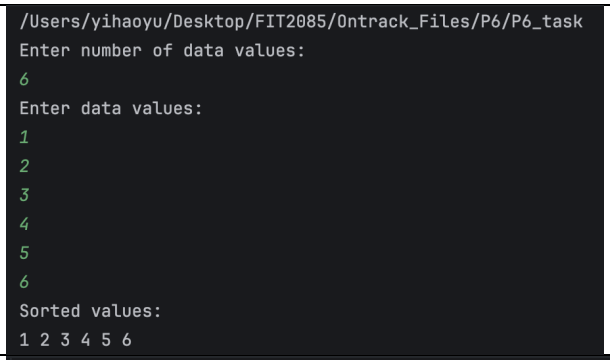
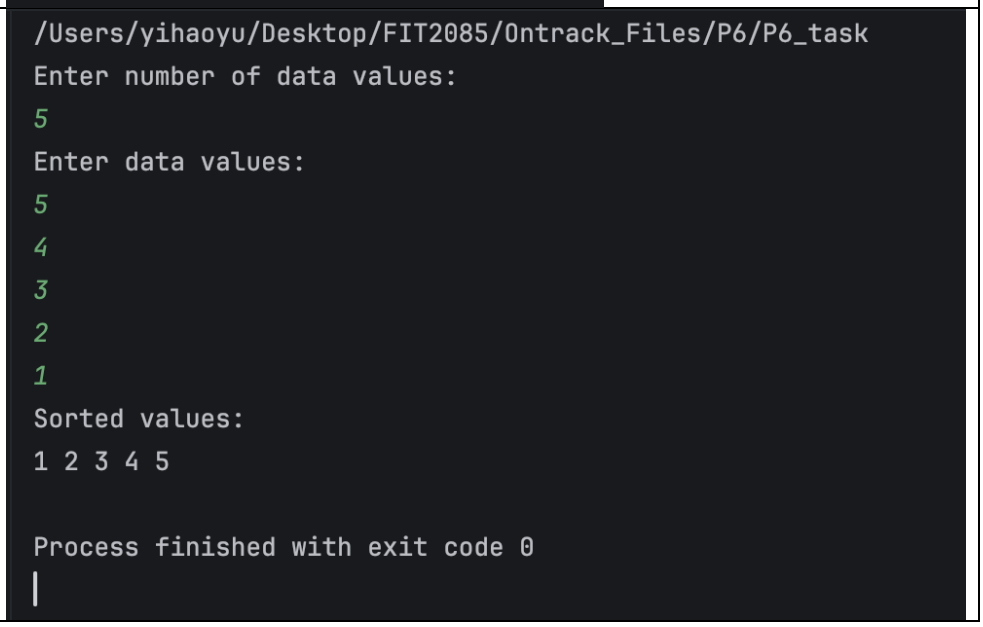
Report Scaffold

Reasoning

Firstly, my algorithm gets the number of available servers, and limits it to n if k>n. The input array is then divided into k sublists as evenly as possible, so their sizes differ by at most 1. Each sublist is recursively sorted by using the same merge_sort function. The base case occurs when the sublist contains only one element. Subsequently, merge_sorted_lists is used to combine the sorted sublists. In the merge_sorted_lists, I merge sort the sublists sequentially: the first list becomes the initial merged result, then each sublists remained is merged with current result using standard two pointer merge technique.

Evidence of Testing

Test Case	Proof
Already sorted	
Reserve order	

Duplication	/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/P6/P6_task Enter number of data values: 5 Enter data values: 1 2 3 3 0 Sorted values: 0 1 2 3 3
Negative numbers	/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/P6/P6_task Enter number of data values: 5 Enter data values: -1 -2 -3 -4 -5 Sorted values: -5 -4 -3 -2 -1

Time Complexity Analysis

Let n be the number of elements and k be the number of available servers at a recursive call. The input is divided into approximately equal sublists of size n/k.
My merging method is sequential. After copying the first sublist, the second merge processing approximately 2n/k elements, the third merge processing approximately 3n/k elements and this continues until the final merge processing n elements.
Therefore:

$T(n, k) = n/k + 2n/k + \ldots + k = n/k * (1+k)*k/2 = n * (1+k) /2$
$= O(nk)$

At every recursion level, it would cost $O(nk+n) = O(nk)$ for splitting and merge_sorted_list.

We have $\log_k n$ recursion levels, therefore:
$O(nk)*O(\log_k n) = O(nk \log_k n)$

Worst case: k=n
$O(nk \log_k n) = O(n^2)$

Best case: k=2
$O(nk \log_k n) = O(n\log n)$

Word Count

This document contains 252 words.